

🔗 Machine Learning Regression Notes (Beginner-Friendly)

1. What is Regression?

Regression is a **Supervised Machine Learning** technique used to predict **continuous numerical values**.

Examples

Problem	Output
House Price Prediction	50,00,000
Salary Prediction	8,50,000
Temperature Forecasting	35°C
Sales Prediction	500 Units

Classification vs Regression

Classification	Regression
Predicts Categories	Predicts Numbers
Spam/Not Spam	House Price
Yes/No	Salary Amount
Survived/Not Survived	Temperature

Dataset Setup

```
import pandas as pd

df = pd.read_csv("house_price.csv")
```

```
X = df.drop("Price", axis=1)
y = df["Price"]
```

2. Linear Regression

Concept

Linear Regression finds the **best fit line** that describes the relationship between input and output.

Equation:

```
genui {"algebra_functions_learning_block":
{"type_id":"GRAPHABLE_FUNCTION","content":"y = mx + b"}}
```

Where:

- y = Predicted Value
 - x = Input Feature
 - m = Slope
 - b = Intercept
-

Example

Suppose:

House Size	House Price
1000 sq ft	20 Lakh
1500 sq ft	30 Lakh
2000 sq ft	40 Lakh

Model learns:

Bigger house Higher price

Simple Linear Regression

One input feature

House Size Price

Multiple Linear Regression

Multiple input features

Size
Bedrooms
Location
Age

House Price

Implementing Linear Regression

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)
```

Prediction

```
predictions = model.predict(X_test)
```

Coefficients

```
print(model.coef_)  
print(model.intercept_)
```

Interpretation

```
coef_      slope (m)  
intercept_ b
```

3. Cost Function

Why do we need it?

Model predictions are never perfect.

We need a way to measure error.

Cost Function tells:

```
How wrong is our model?
```

Error

Error = Actual - Predicted

Example:

Actual Price = 50

Predicted Price = 45

Error = 5

Mean Squared Error (MSE)

Most common cost function in Linear Regression.

Formula:

[$MSE = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2$]

Why Square Errors?

Without square:

+5 and -5 cancel each other

With square:

25 and 25

No cancellation.

Large mistakes get punished more.

4. Gradient Descent

Concept

Gradient Descent helps find the best values of:

m (slope)
b (intercept)

that minimize the cost function.

Process

Step 1

Make random guess

m = 2
b = 5

Step 2

Calculate Cost

Step 3

Adjust m and b

Step 4

Repeat until minimum cost

Learning Rate

Controls step size.

Too Small Slow Learning

Too Large Overshoot Minimum

Good Value Faster Convergence

Visual Understanding

Top of Hill



Minimum Cost

Gradient Descent keeps moving downhill.

5. Train-Test Split

Why Needed?

If we train and test on same data:

Model memorizes data

Not real learning.

Standard Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,
```

```
y,  
test_size=0.2,  
random_state=42  
)
```

Typical Ratios

Train Test

80% 20%

70% 30%

75% 25%

Complete Linear Regression Example

```
import pandas as pd  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
  
df = pd.read_csv("house_price.csv")  
  
X = df[["Area"]]  
y = df["Price"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)  
  
model = LinearRegression()  
  
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

6. Model Evaluation Metrics

Mean Squared Error (MSE)

Formula:

$$[\text{MSE} = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2]$$

Code:

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(
    y_test,
    pred
)

print(mse)
```

Root Mean Squared Error (RMSE)

Square root of MSE.

Formula:

$$[\text{RMSE} = \sqrt{\text{MSE}}]$$

Code:

```
from sklearn.metrics import mean_squared_error
import numpy as np

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        pred
    )
)

print(rmse)
```

Why RMSE?

RMSE is easier to understand because it is in the same unit as target.

Example:

```
House Price Error = 2 Lakh
```

R² Score (Coefficient of Determination)

Measures how well model explains data.

Formula:

$$[R^2 = 1 - \frac{SS_{\text{residual}}}{SS_{\text{total}}}]$$

Interpretation

R ² Score	Meaning
1.0	Perfect Model
0.9	Excellent
0.8	Very Good

0.5	Average
0	Poor
<0	Worse than Mean Prediction

Code

```
from sklearn.metrics import r2_score

score = r2_score(
    y_test,
    pred
)

print(score)
```

7. Underfitting

What is Underfitting?

Model is too simple.

Cannot learn patterns.

Visual

Actual Pattern

```
*  *
*  *
*  *
 *
```

Model



Straight line misses pattern.

Characteristics

High Training Error

High Testing Error

Causes

- Model too simple
 - Not enough features
 - Insufficient training
-

8. Overfitting

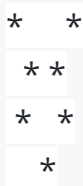
What is Overfitting?

Model memorizes training data.

Fails on new data.

Visual

Data



Model

\\\\\\\\\\\\\\

Curve passes through every point.

Characteristics

Very Low Training Error

High Testing Error

Causes

- Too complex model
- Small dataset
- Too many features

Good Fit

Training Error Low
Testing Error Low

Best scenario.

Comparison

Type	Train Error	Test Error
Underfitting	High	High
Good Fit	Low	Low
Overfitting	Very Low	High

9. House Price Prediction Project

Goal

Predict house prices using Linear Regression.

Features

Area
Bedrooms
Bathrooms
Age
Location Score

Target:

Price

Step 1: Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    mean_squared_error,
    r2_score
)
```

Step 2: Load Data

```
df = pd.read_csv("HousePrice.csv")
```

Step 3: Features and Target

```
X = df.drop("Price", axis=1)
```

```
y = df["Price"]
```

Step 4: Split Data

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

Step 5: Train Model

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Step 6: Predict

```
pred = model.predict(X_test)
```

Step 7: Evaluate

```
mse = mean_squared_error(y_test, pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, pred)

print("MSE :", mse)
print("RMSE:", rmse)
print("R2 :", r2)
```

Interview Revision (1-Minute)

Regression

- Predicts numerical values
- Supervised Learning

Linear Regression

- Best Fit Line
- Equation: $y = mx + b$

Cost Function

- Measures error
- MSE most common

Gradient Descent

- Minimizes cost
- Updates weights

Train-Test Split

- Prevents memorization
- Tests generalization

Metrics

- MSE Average Squared Error
- RMSE Error in original units
- R^2 Model performance score

Underfitting

- Model too simple

Overfitting

- Model too complex